AI CHATBOT PLATFORM

Project Scope, Technical Plan & Development Roadmap

Version 1.2 | Planning document | 23 October 2026

Purpose: define the product, architecture, delivery sequence, security controls, and release criteria before implementation begins.

Document Status & How to Use This Plan

This is a pre-development specification, not evidence that the platform has already been implemented or tested. Items marked as recommendations should be confirmed before coding. Keep the document updated when a decision changes.

Label	Meaning
Confirmed direction	A direction already agreed in project planning.
Recommendation	A practical default proposed for the MVP; confirm before implementation.
Open decision	A choice that must be resolved before the affected feature is built.
Release gate	A condition that must pass before moving to the next stage or going live.

Primary rule Build a small, secure product that can serve the first 3–5 paying clients reliably. Do not build every advanced feature before validating demand.

1. Executive Summary

Build a multi-tenant AI website assistant platform for service-based businesses. A platform administrator will onboard each business, enter and approve its public business information, configure a chatbot, generate an embed snippet, and monitor conversations, leads, usage, and errors. A lightweight JavaScript widget will appear on each client website. The shared backend will resolve the correct tenant, retrieve only that tenant's published knowledge, call the configured local Ollama model, and return a grounded answer.

The first release prioritizes manual onboarding and reliable FAQ/lead-generation use cases. Automatic crawling, client self-service, subscription billing, and complex integrations are intentionally deferred.

2. Product Goals, Users & Boundaries

2.1 Goals

- One shared backend supports multiple businesses with strict tenant data isolation.
- A business can install the widget without custom backend code for each client.
- Answers rely on business-approved content; unknown facts, prices, guarantees, and policies must not be invented.
- Interested visitors can submit leads, and the correct business can view them.
- The admin can see basic usage, errors, latency, and activity.
- Validate the offer with 3–5 paying clients before investing in broad automation.

2.2 Initial target market

Service businesses with primarily informational websites: web-development agencies, digital-marketing/SEO agencies, software companies, consulting firms, recruitment/staffing providers, and similar organizations.

2.3 Roles

Role	V1 access
Platform Admin	Create/manage clients, approve content, configure bots, view all platform usage and leads, troubleshoot.
Client User	Not required in V1. Planned later with strict access limited to their own tenant.
Website Visitor	Ask questions and optionally submit clearly explained contact details.

2.4 Out of scope for V1

- E-commerce product catalogue, inventory, checkout, order tracking, or customer-account integrations.
- Mandatory automatic website crawling or automatic publishing of imported content.
- Client self-registration, self-service billing, automated invoices, and payment retries.
- Unlimited messages or guaranteed unlimited concurrency.
- A separate backend or model runtime for every client by default.

3. Confirmed Direction & Open Decisions

Area	Current position	Status
Product	Own platform, not a white-label reseller product	Confirmed
Architecture	Shared multi-tenant backend	Confirmed
Backend/database	Node.js + Express; MongoDB + Mongoose	Proposed baseline
Model runtime	Ollama with qwen2.5:3b for initial testing	Current choice; benchmark before launch
Content onboarding	Manual entry and approval in V1	Confirmed
Widget	Embeddable JavaScript widget	Confirmed
Dashboard	Admin-only for V1	Recommended
Knowledge retrieval	Structured records + tenant-scoped search; add chunking/embeddings only if needed	Technical design required
Usage limit	Track messages and conversations; final plan limits not set	Open
Retention/consent	Collect minimum lead data and define retention/notice	Open before launch
Email delivery	May be deferred; dashboard + WhatsApp/contact link are V1 options	Open
Pricing/domain/name/VPS sizing	Not finalized	Open

4. System Architecture

Logical request path:

Component	Responsibility
Client website	Loads the widget snippet over HTTPS.
JavaScript widget	Displays chat, sends messages, presents safe responses and lead form.
Public Chat API	Validates payload and origin policy, rate-limits, resolves tenant, checks status/quota.
Knowledge service	Retrieves only published information belonging to the resolved tenant.
Conversation service	Stores messages and retrieves only the authorized session's history.
LLM adapter	Calls Ollama, applies timeouts, normalizes errors, and permits future model changes.
Lead service	Validates lead fields, stores lead against tenant and conversation, returns configured next step.
Admin API/dashboard	Requires admin authentication; manages clients, content, configuration, leads and metrics.

MongoDB	Stores tenant-owned data with tenant-scoped queries and appropriate indexes.
Ollama runtime	Private internal service; should not be directly exposed to the public internet.

Important security distinction A tenant/client ID included in widget code is public and is not proof of authorization. Domain checks are useful as one signal but must not be the only security control.

4.1 Recommended deployment shape

- Serve the dashboard, API, and widget assets over HTTPS behind a reverse proxy.
- Keep MongoDB and Ollama private; allow access only from required application services.
- Run the API and Ollama under managed services with restart policies and health checks.
- Store secrets in server environment/configuration, never in widget JavaScript or client-visible responses.
- Set up automated database backups and periodically test restoring them.

5. Technology Plan

Layer	Initial choice	Implementation note
Admin UI	React or Next.js	Prefer the existing project conventions; choose one before implementation.
API	Node.js + Express	Separate admin routes and public widget routes.
Database	MongoDB + Mongoose	Tenant ID on all tenant-owned records; add indexes for common queries.
Model	Ollama / qwen2.5:3b	Keep configurable behind a model adapter; benchmark on the actual VPS.
Widget	Vanilla JavaScript or small bundled widget	Keep lightweight, responsive, isolated from host-page CSS.
Hosting	VPS + reverse proxy + HTTPS	Capacity must be measured, not assumed.

6. Business & Content Onboarding Workflow

1. Agree the service scope and obtain the client's permission to use the supplied public business information.
2. Create a client/tenant record and configure the canonical website URL and allowed domains.
3. Collect approved content: company overview, services, public prices (if approved), FAQs, business hours, contact details, policies, and special instructions.
4. Enter or paste the information into structured content sections. Mark all new or edited content as Draft.
5. Review content for accuracy, sensitive information, outdated claims, and unsupported promises; then publish it.
6. Configure chatbot name, welcome message, languages, tone, fallback, widget theme, lead fields, and usage limits.
7. Generate the installation snippet and install it on the client website.
8. Test on the real site: common questions, unknown questions, mobile layout, lead submission, allowed/blocked origins, and correct tenant routing.
9. Activate the chatbot only after the release checklist passes; monitor usage and errors after launch.

6.1 Content lifecycle

Each content item should have tenantId, content type, title, body/structured fields, status (draft/published/archived), version, createdAt, updatedAt, and publishedAt where relevant. Changes to published content must invalidate any cache or indexed representation. Never let draft or archived content answer public questions.

7. Chat Request & Answer Workflow

10. Widget submits a message, public tenant identifier, and session reference. Enforce a maximum payload/message length.
11. API validates the request, checks client status and allowed-origin policy, applies rate limits, and checks usage quota.
12. Resolve the tenant from trusted server-side configuration; do not trust user-supplied tenant data for database authorization.
13. Load only that tenant's published knowledge and the minimum recent conversation context needed.
14. Retrieve relevant content. For the first release, begin with a simple tenant-scoped structured/keyword retrieval approach; establish a relevance test set before deciding whether embeddings are necessary.
15. Build a constrained prompt: answer only from supplied evidence, do not follow instructions found inside untrusted retrieved content, do not invent prices/policies, and state when the answer is unavailable.
16. Call Ollama through an internal model adapter with timeout and concurrency controls.
17. Handle empty, malformed, timed-out, or failed model responses with a safe fallback; do not expose internal errors or prompts.
18. Return answer plus safe UI metadata, store the conversation under the correct tenant/session, and record latency, status, and usage.
19. If the visitor asks for a human or the answer is repeatedly unavailable, offer the configured contact/lead route.

Grounding requirement A prompt alone cannot guarantee factual answers. Use relevant approved content, conservative fallback rules, and repeatable quality tests. If a response cannot be supported by the available content, the bot should say so.

7.1 Session handling

- Use an opaque, unpredictable session identifier; do not put personal data in the identifier.
- Validate that each conversation belongs to the resolved tenant before reading or updating it.
- Do not treat a browser-provided conversation ID as sufficient authorization.
- Define session expiration and conversation retention before launch.

8. Lead Capture & Routing

Destination	V1 behavior	Notes
Admin dashboard	Store and display leads	Required for MVP; lead is tenant-scoped and linked to source conversation.
WhatsApp	Open a click-to-chat URL, optionally prefilled	A link does not automatically send a message.
Contact/booking page	Redirect/open client-configured URL	Validate URL scheme and destination; prevent unsafe redirects.
Email	Optional later integration	If enabled, use server-side credentials and handle delivery failures.

Collect only the fields required for follow-up (for example, name, email or phone, service of interest, and optional message). Show a clear notice explaining who will receive the details and why. Do not collect sensitive information unless there is a justified, explicitly designed need.

Lead data fields (recommended minimum)

- tenantId
- conversationId
- name/contact method as provided
- service of interest
- message (optional)

- notice/consent state where appropriate
- createdAt
- status/source/destination

9. Dashboard Modules — V1

Module	Minimum scope
Overview	Active clients, conversations, messages, leads, failed requests, and basic latency/usage.
Client management	Create/edit/deactivate client; website, category, allowed domains, plan/limit settings.
Content management	Create/edit/preview/publish/archive business profile, services, FAQs, pricing and policies.
Chatbot settings	Name, greeting, tone, language behavior, fallback, widget theme and lead destinations.
Widget setup	Generate snippet; show installation instructions and test/status indicator.
Conversations	Restricted support/debug view with tenant, time and status filters.
Leads	View/filter leads; update follow-up status; export is optional and should be access-controlled.
Usage & errors	Messages/conversations, quota usage, failures, latency and last activity.
System health	API/model/database availability and basic operational status; never display secrets.

10. Data Model — Conceptual

Collection	Core fields / notes
Client/Tenant	name, category, websiteUrl, allowedDomains, status, limits/plan, timestamps
BusinessContent	tenantId, type, title, body/fields, status, version, timestamps
ChatbotConfig	tenantId, welcomeMessage, languages, tone, fallback, widgetTheme, leadSettings
Conversation	tenantId, opaque session reference, source domain, messages, createdAt, lastActivityAt
Lead	tenantId, conversationId, approved contact fields, status, notice/consent state, createdAt
UsageCounter/UsageRecord	tenantId, period, messageCount, conversationCount, failures, latency aggregates
AdminUser	identity, password hash or trusted auth reference, role, status, timestamps
AuditLog	actor, action, tenantId if applicable, timestamp, safe metadata

This is a conceptual model, not a final schema. Specify indexes, validation rules, deletion behavior, and retention before coding. Avoid storing duplicate or unnecessary personal data.

11. API Design Areas

Area	Example responsibility	Access
Admin authentication	Login/logout/session, current admin	Admin only

	identity	
Clients	Create/update/status/domain settings	Admin only
Content	Create/edit/publish/archive tenant content	Admin only in V1
Chatbot config	Read/update widget settings	Admin write; public widget receives only safe public settings
Public chat	Validate widget request, retrieve content, call model, return answer	Public, rate-limited
Conversations	Read support history and store messages	Admin-only read in V1; public session writes validated
Leads	Capture/list/update status	Public create with strict validation; admin read/update
Usage/health	Quota checks, counters, health metrics	Internal/admin; expose only minimal public status

Before implementation, define exact endpoint paths, request/response schemas, authentication, validation, error codes, rate limits, and pagination. Never accept tenantId from a public request as the sole scope for a database query.

12. Version Roadmap & Build Order

Version 1 — MVP for first paying clients

20. Project foundation: repository structure, environment config, error handling, health checks, database connection and logging.
21. Admin authentication and protected admin API.
22. Tenant/client management, domain configuration, activation status and usage limits.
23. Content CRUD with draft/published/archive lifecycle.
24. Chatbot settings and safe public configuration response.
25. Tenant-scoped retrieval and Ollama adapter with timeouts, fallback and concurrency control.
26. Embeddable widget with loading/error states, mobile layout and session handling.
27. Conversation storage, lead capture, dashboard lead list, WhatsApp/contact routing.
28. Usage counters, quota enforcement, basic analytics and operational monitoring.
29. Security, quality, load, recovery and real-site installation tests before launch.

V1 release gates

- A new client can be onboarded without custom backend code.
- Only published content from the correct tenant is used for an answer.
- Unknown prices/services/policies are not fabricated in the agreed test set.
- Changing tenant IDs, conversation IDs, or request parameters cannot reveal another tenant’s data.
- Rate limits and quotas are enforced server-side.
- Ollama timeout/offline, database failure, malformed inputs and invalid origins fail safely.
- Backup restoration and production deployment/recovery steps are tested.
- Target-VPS latency and concurrency are measured and accepted before onboarding clients.

Version 2 — Automation & client self-service

- Website crawling/import with page selection, crawl status and duplicate handling.
- Admin review and approval before imported content is published.
- Restricted client portal for their own leads, conversations and usage.
- Email notifications and improved lead follow-up.
- Analytics for unanswered questions, lead conversion, latency and usage trends.
- Improved retrieval (chunking, search and embeddings/vector search if tests show value).
- Content refresh, stale-content indicators, monitoring, alerts and audit logs.
- More widget customization without code changes.

Version 3 — Commercial scale & integrations

- Subscription plans, payment integration, invoices, renewals and failed-payment handling.
- Guided client self-onboarding.
- CRM, webhook, booking and other integrations prioritized from paying-client demand.
- More advanced quotas, abuse protection, queueing, caching and scaling.
- Model comparison based on measured answer quality, latency and infrastructure cost.
- Data export/deletion, retention controls and stronger privacy administration.

Future / not committed

- E-commerce product search, inventory, checkout, order tracking and authenticated customer workflows.
- Voice assistants and complex integrations with private client systems.
- Dedicated infrastructure per tenant, unless customer requirements and economics justify it.

13. Security, Privacy & Abuse Controls

- Require authentication on every admin endpoint; store passwords only as secure hashes if using password authentication.
- Apply tenant scope on every tenant-owned read, update, delete, export and background job; add automated cross-tenant tests.
- Keep model runtime, database and internal endpoints private; expose only required API routes.
- Use HTTPS, secure headers, server-side secrets, input validation, payload limits and safe error messages.
- Use rate limiting by multiple signals where practical (IP, tenant, session); account for trusted reverse-proxy configuration.
- Treat website content and visitor messages as untrusted input. Test prompt injection; never allow retrieved text to override system policy or authorize actions.
- Restrict CORS/origin policy appropriately; remember browser origin checks do not replace authentication or abuse protection.
- Minimize logs containing message content and personal data; protect access to conversation/lead records.
- Define retention, deletion, backup access and incident-response steps before launch.
- Validate redirect URLs and never allow user input to select internal/private network destinations.

14. Testing Plan

Test group	Required cases
Functional	Create client, publish content, configure bot, install widget, chat, capture lead, update status.
Knowledge quality	Known FAQ, missing information, pricing, ambiguity, conflicting content, Bangla/Banglish and English.
Tenant isolation	Try to read/update another tenant's content, conversation, lead, configuration and usage.
Widget/API abuse	Invalid domain, altered tenant ID, oversized message, malformed JSON, repeated requests, invalid URLs.
LLM failures	Ollama offline, timeout, empty response, malformed output, overload and slow response.
Database failures	Connection failure, failed write, unavailable database, retry behavior and no duplicate lead.
Privacy	Notice shown, minimum fields, authorized access, retention/deletion behavior.
Performance	Median and high-percentile latency, concurrency, queueing, CPU/RAM and error rate on intended VPS.
Regression	Repeat the same fixed evaluation set after prompt, model, retrieval or config changes.

Keep a small version-controlled evaluation dataset with expected evidence/fallback outcomes. Model quality should be judged against repeatable cases, not a few informal conversations.

15. Deployment & Operations

30. Choose and document the production domain, VPS size, operating system, and deployment approach.
31. Create production environment variables and unique secrets; never reuse development credentials.
32. Configure reverse proxy and HTTPS; restrict database and Ollama ports to private access.
33. Run API/model services under managed processes with restart policies and health checks.
34. Configure MongoDB users with least privilege, automated backups, and a tested restore procedure.
35. Monitor disk, memory, CPU, uptime, API errors, model timeouts and latency.
36. Deploy widget assets over HTTPS and verify on a real client site, including mobile.
37. Run security and load tests before enabling production clients.
38. Document deploy/rollback, backup restore, client offboarding, content correction and incident handling.

16. Risks & Mitigations

Risk	Mitigation
Incorrect or outdated answers	Approved content, conservative fallback, content review, fixed evaluation set.
Cross-tenant leakage	Tenant-scoped data access everywhere; automated negative authorization tests.
Slow answers under load	Benchmark on target VPS, cap concurrency, queue, limit context and scale based on evidence.
High manual support effort	Standard onboarding questionnaire, reusable templates and clear support scope.
Uncontrolled usage	Server-enforced quotas, rate limits, usage counters and fair-use terms.
Widget abuse	Payload limits, rate limits, origin policy, monitoring and abuse response.
Model limitations	Keep model adapter replaceable and benchmark alternatives against the same evaluation set.
Sensitive lead data exposure	Data minimization, access controls, retention/deletion policy and safe logging.

17. Pricing & Commercial Operations (Not Finalized)

Pricing is intentionally not locked in this technical document. Before selling, estimate hosting and inference capacity, expected messages per client, support/onboarding time, backup/monitoring costs, and margin. Define a clear setup fee, monthly fee, message/conversation allowance, overage policy, support scope, and what happens when a client is inactive or exceeds a quota. Do not advertise unlimited use until capacity and cost are proven.

Initial manual onboarding and manual payment collection can be used in V1; automate billing only after the service and pricing are validated.

18. Definition of Done for V1

- Admin can create a client, enter/publish approved content, configure the chatbot and generate the widget snippet.
- Widget works on configured client websites and has usable loading, timeout, offline and fallback states.
- Responses use the correct tenant's published content; unsupported facts trigger the configured fallback.

- Leads and conversations are stored and displayed under the correct tenant.
- Usage is measured consistently and limits are enforced server-side.
- Admin routes are protected; cross-tenant access and common widget abuse tests pass.
- HTTPS, monitoring, backup/restore and recovery notes are in place.
- Quality and performance results on the intended VPS are documented and accepted.
- At least one end-to-end pilot on a real website has been completed before wider sales.

19. Pre-Development Checklist

Decision / task	Why it must be settled
Choose React vs Next.js for dashboard	Avoid starting with an unresolved frontend architecture.
Define exact V1 content types and fields	Keeps content UI, retrieval and tests consistent.
Choose first retrieval method and evaluation set	Prevents adding vector infrastructure without evidence.
Define message/conversation quota semantics	Avoids inconsistent counting and unclear pricing.
Define lead fields, notice and retention	Protects visitors and sets a clear data policy.
Choose VPS and measure Ollama capacity	Determines realistic latency and concurrency.
Define widget origin and tenant resolution strategy	Reduces spoofing and cross-tenant risks.
Write API contracts and error codes	Lets frontend/backend development proceed consistently.
Agree launch checklist and rollback plan	Avoids untested production changes.

Recommended next step Before coding, turn Sections 4–11 into a short technical design: repository layout, data schemas, API contracts, authentication/session approach, tenant resolution, and the first knowledge-quality test set. Then implement V1 in the numbered order in Section 12.

Appendix A — Working Assumptions

- This document assumes an admin-operated platform for service-business FAQ and lead generation.
- The current Ollama model is a starting point, not a promise of production quality or capacity.
- Pricing, final limits, retention periods, product name/domain, and exact VPS sizing remain open.
- Requirements may change after real-client feedback; changes should be recorded with date and rationale.

Appendix B — AI Coding Agent Development & Review Workflow

The implementation may be carried out with Antigravity or another AI coding assistant. The coding agent is responsible for making the requested code changes and running the available checks; the project owner then shares the agent's report, diffs, logs, screenshots, and test results with ChatGPT for an independent review. Work must proceed in small, reviewable steps rather than asking the coding agent to build the entire platform in one prompt.

Mandatory loop Plan one step → implement only that step → run checks → report exact evidence → review the result → approve or request fixes → move to the next step. Do not skip the review gate after a step.

B1. Ground rules for Antigravity / any coding agent

- Read the current project instructions and inspect the repository before editing. Do not assume the existing structure.
- Do not rewrite unrelated files, upgrade dependencies, or change architecture without explaining the need and obtaining approval.
- Do not claim a test passed unless it was actually executed; include the exact command and result.
- Never paste production secrets, private keys, real customer data, or credentials into prompts, logs, screenshots, or commits.
- Use small changes with clear scope. Keep code formatting and existing conventions consistent.
- Add or update tests for behavior changed in the step. If a test cannot run, explain why and provide the closest available evidence.
- After each step, summarize files changed, decisions made, tests run, failures remaining, and how to reproduce the result.
- Do not start the next step until the current step has been reviewed and approved, except for a specifically authorized fix round.

B2. Step-by-step implementation plan

Step	Scope / deliverable	Evidence to send for review	Review gate
0. Repository audit & plan	Inspect repos, package files, current app behavior, environment needs, test commands and deployment assumptions. No feature code yet.	Repo tree, package scripts, architecture summary, current test/build status, risks and proposed file plan.	Approve architecture and confirm no hidden existing functionality is being overwritten.
1. Project foundation	Environment validation, config separation, error handling, logging/request IDs, health endpoint, database connection lifecycle.	Changed-file list, env example with no secrets, exact test/build commands and outputs.	App starts safely; missing config fails clearly; secrets are not logged.
2. Admin authentication	Secure admin login/session approach and middleware protecting admin APIs.	Auth flow, route list, tests for unauthenticated/invalid access; no real credentials.	Protected routes reject unauthorized requests; no secret exposure.
3. Tenant/client management	Create/update/deactivate client, allowed domains, public widget identifier, per-tenant limits.	Schema, routes, validation, test cases for active/inactive clients and invalid input.	Tenant resolution design approved; public identifier is not treated as a secret.
4. Content management	Structured content CRUD, draft/published/archive lifecycle, tenant scope and cache/index invalidation plan.	Schema and API examples, UI screenshots, tests for publication state and cross-tenant access.	Only published content for the correct tenant is available to chat.
5. Chatbot configuration	Per-tenant greeting, tone, language, fallback, widget theme and lead settings; safe public config endpoint.	Config schema, endpoint response example, tests proving secrets/internal fields are excluded.	Public config contains only fields safe to expose.
6. Knowledge retrieval & model adapter	Tenant-scoped retrieval, bounded context, Ollama call with timeout/concurrency	Retrieval examples, model settings, fixed evaluation questions/results,	Unknown facts trigger fallback; prompt injection and tenant leakage tests pass at

	handling, safe fallback.	timeout/offline tests.	the agreed baseline.
7. Public chat API & sessions	Input validation, rate limits, session handling, tenant checks, message storage and usage counting.	Request/response examples, error codes, tests for altered IDs, oversized input, repeated requests.	Changing tenant/session IDs cannot access another tenant's data; quotas enforced server-side.
8. Embeddable widget	Responsive widget, loading/error states, safe rendering, allowed-domain checks and install snippet.	Widget screenshot/video, snippet example, real-site test steps, build output.	Works on a test client site without exposing secrets or breaking host-page layout.
9. Leads & routing	Lead form, minimum required fields, notice text, tenant-scoped storage, dashboard list, WhatsApp/contact routing.	Lead flow screenshots, API examples, validation and duplicate/failure tests.	Lead tied to correct tenant/conversation; safe URL handling; notice visible.
10. Usage & admin views	Dashboard overview, conversations, leads, usage/errors, basic filters and status.	Screenshots, role/access tests, sample aggregate data with no personal data.	Admin sees needed metrics; tenant-specific records remain scoped.
11. Hardening & acceptance tests	Security regression, knowledge evaluation, model/database failure cases, load and latency tests.	Full test report with commands, pass/fail, unresolved issues, target VPS specs and metrics.	All V1 release gates pass or remaining risks are explicitly accepted before pilot.
12. Production deployment & pilot	HTTPS, private ports, production secrets, backup/restore, monitoring, rollback notes and one real-site pilot.	Deployment checklist, health checks, restore evidence, redacted logs, pilot feedback.	No launch until recovery and security checks are verified.

B3. Standard review packet after every step

Ask the coding agent to return all of the following after completing a step:

- Step number and exact scope completed.
- Files created/modified/deleted, with a short explanation for each.
- Important implementation decisions and any deviations from the specification.
- Exact commands executed (tests, lint, type-check, build) and their real outputs/status.
- Tests added/updated, including negative/security cases.
- Screenshots or request/response examples where UI/API behavior changed.
- Known issues, skipped checks, assumptions, and remaining risks.
- Confirmation that unrelated code and dependencies were not changed, or a justification if they were.

B4. Prompt to give the coding agent for each step

You are implementing Step [N] of the AI Chatbot Platform specification. First inspect the current repository and relevant files. Do not implement future steps. Follow the existing conventions and the agreed architecture. Before editing, briefly state your plan and the files you expect to change. Implement only this step, add/update tests for the changed behavior, and run the relevant tests/lint/type-check/build commands available in the repository. Do not claim checks passed unless you ran them. Do not change unrelated files, upgrade dependencies, or alter architecture without explaining why. Do not expose secrets or real customer data. At the end, provide: (1) summary, (2) files changed, (3) commands and exact results, (4) test coverage including negative cases, (5) screenshots/request-response examples if relevant, (6) known issues and skipped checks, and (7) any decisions that need approval. Stop after this step and wait for review.

Use Replace [N] with the approved step number and attach the relevant sections of this specification. Do not give the agent all future tasks as permission to implement them at once.

B5. Prompt to request an independent ChatGPT review

Review this implementation as a strict senior engineer and security reviewer against the AI Chatbot Platform specification and the acceptance criteria for Step [N]. Do not assume the coding agent's report is correct. Inspect the actual diff/files, test output, logs and screenshots I provide. Separate verified facts from claims and missing evidence.

Check correctness, maintainability, tenant isolation, authentication/authorization, input validation, privacy, failure handling, tests and regressions relevant to this step. Return: (1) verdict: PASS / PASS WITH REQUIRED FIXES / FAIL, (2) blocking issues ordered by severity with file/line references where available, (3) missing tests or evidence, (4) non-blocking improvements, (5) exact follow-up prompt for the coding agent to fix only the blocking issues, and (6) a short retest checklist. Do not approve the next step while any blocking issue remains.

B6. Review verdict rules

Verdict	Meaning	Next action
PASS	Acceptance criteria met and adequate evidence provided; no blocking issue found in the reviewed scope.	Record approval and move to the next step.
PASS WITH REQUIRED FIXES	Core direction is sound, but one or more required fixes or missing tests remain.	Send only the fix prompt; rerun relevant tests and request another review.
FAIL	Major correctness, security, architecture, or evidence gap; implementation cannot be trusted yet.	Stop progression, correct the design/implementation, then repeat review.

B7. Final review before pilot

- Review the complete change set, not only the latest step.
- Run all backend/frontend tests, lint/type checks and production builds from a clean, documented state.
- Run cross-tenant negative tests for content, config, conversations, leads and usage.
- Test Ollama unavailable/slow, database unavailable, malformed input, rate-limit exceeded and quota exceeded.
- Run the fixed knowledge evaluation set in every supported language promised to clients.
- Verify no secrets, private customer data or development-only settings are committed or exposed.
- Verify HTTPS, private service ports, backup restoration, monitoring and rollback instructions.
- Record known limitations and get explicit approval for any remaining accepted risks.